

Devoir Pratique

Développement d'API REST CRUD avec Spring Boot

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant devra être capable de :

- Créer un projet Spring Boot.
- Concevoir une entité JPA.
- Configurer une base de données avec Spring Data JPA.
- Développer une API REST.
- Réaliser les opérations CRUD.
- Effectuer des contrôles métiers dans le contrôleur REST.
- Retourner des codes HTTP adaptés selon les erreurs rencontrées.
- Documenter une API avec Swagger/OpenAPI.
- Tester une API à l'aide de Swagger UI.

Contexte

L'ISEP-AT souhaite développer une application permettant de gérer les étudiants .

Chaque étudiant est identifié par un **matricule unique** et possède les informations suivantes :

- Matricule
- Prénom
- Nom
- Email
- Date de naissance
- Lieu de naissance
- Nationalité

L'application devra permettre :

- d'ajouter un étudiant ;

- de modifier un étudiant ;
- de supprimer un étudiant ;
- de rechercher un étudiant par son identifiant ;
- d'afficher la liste des étudiants.

Remarque : l'âge ne sera pas stocké dans la base de données. Il sera calculé et affiché uniquement dans l'application cliente (Front-end).

I. Création du projet

Créer un projet Spring Boot avec les dépendances suivantes :

- Spring Web
- Spring Data JPA
- MySQL ou PostgreSQL
- Lombok
- Springdoc OpenAPI (Swagger)

II. Configuration

Configurer le fichier **application.properties**.

- créer un profil dev qui utilise une base de données isepat_dev
- créer un profil prod qui utilise une base de données isepat_cloud

III. Création de l'entité

Créer l'entité **Etudiant**.

Attribut	Type	Contraintes
id	Long	Clé primaire
matricule	String	Obligatoire, unique
prenom	String	Obligatoire
nom	String	Obligatoire
email	String	Obligatoire, unique
dateNaissance	LocalDate	Obligatoire
lieuNaissance	String	Obligatoire
nationalite	String	Obligatoire

Travail demandé

À partir des contraintes ci-dessus :

- créer l'entité JPA ;
- définir la clé primaire ;
- configurer la génération automatique de l'identifiant ;
- appliquer les contraintes sur les colonnes concernées.

IV. Repository

Créer l'interface

`EtudiantRepository`

héritant de

`JpaRepository<Etudiant, Long>`

Ajouter également les méthodes nécessaires pour vérifier :

- l'existence d'un matricule ;
- l'existence d'un email.

V. Service

Créer la classe

`EtudiantService`

Elle devra proposer les opérations suivantes :

`ajouter()`

`modifier()`

`supprimer()`

`rechercher()`

`lister()`

VI. Contrôleur REST

Créer un contrôleur REST exposant les services suivants.

Méthode	URL	Description
POST	/etudiants	Ajouter un étudiant
GET	/etudiants	Lister les étudiants

GET	/etudiants/{id}	Rechercher un étudiant
PUT	/etudiants/{id}	Modifier un étudiant
DELETE	/etudiants/{id}	Supprimer un étudiant

VII. Contrôles dans le contrôleur

À ce stade du cours, les annotations de validation (`@Valid`, `@NotBlank`, `@Email`, etc.) n'ont pas encore été étudiées.

Les contrôles devront donc être réalisés **manuellement dans le contrôleur REST**.

Avant toute insertion ou modification, vérifier les points suivants :

- le matricule est renseigné ;
- le prénom est renseigné ;
- le nom est renseigné ;
- l'email est renseigné ;
- la date de naissance est renseignée ;
- le lieu de naissance est renseigné ;
- la nationalité est renseignée ;
- le matricule n'existe pas déjà ;
- l'email n'existe pas déjà.

VIII. Gestion des erreurs

Votre API devra retourner des codes HTTP adaptés.

Situation	Code HTTP attendu
Création réussie	201 Created
Modification réussie	200 OK
Suppression réussie	204 No Content
Champ obligatoire manquant	400 Bad Request
Matricule déjà existant	409 Conflict
Email déjà existant	409 Conflict
Étudiant introuvable	404 Not Found

Le corps de la réponse devra contenir un message explicite.

Exemples :

```
{"code":400 msg="Le matricule est obligatoire."}
```

IX. Documentation Swagger

Documenter l'ensemble des services REST avec Swagger/OpenAPI.

Pour chaque opération, préciser :

- son objectif ;
- les paramètres attendus ;
- les codes de retour possibles ;
- une courte description de chaque code.

X. Tests

Tester les services à l'aide de Swagger UI.

Test 1 : Ajout

```
{  
  "matricule": "ET001",  
  "prenom": "Moussa",  
  "nom": "Diallo",  
  "email": "moussa@universite.sn",  
  "dateNaissance": "2003-04-15",  
  "lieuNaissance": "Thiès",  
  "nationalite": "Sénégalaise"  
}
```

Résultat attendu

201 Created

Test 2 : Matricule déjà existant

Créer un deuxième étudiant avec le même matricule.

Résultat attendu

409 Conflict

Test 3 : Email déjà existant

Créer un deuxième étudiant avec le même email.

Résultat attendu

409 Conflict

Test 4 : Champ obligatoire

Envoyer une requête avec le prénom vide.

Résultat attendu

400 Bad Request

Test 5 : Recherche

Rechercher un étudiant inexistant.

GET /etudiants/100

Résultat attendu

404 Not Found

Test 6 : Suppression

Supprimer un étudiant existant.

Résultat attendu

204 No Content

Travail à rendre

L'étudiant devra fournir :

- le projet Spring Boot complet ;
- le code source commenté ;
- les captures d'écran de Swagger montrant les différents tests réalisés.

Critères d'évaluation

Critère	Points
Création de l'entité JPA	10
Repository	5
Service métier+Contrôles des données	25
Contrôleur REST	5

Gestion correcte des codes HTTP	20
Documentation Swagger	25
Qualité du code (architecture, lisibilité, bonnes pratiques)	10
Total	100 points

Bonus (+2 points)

Ajouter les fonctionnalités suivantes :

- recherche d'un étudiant par matricule ;
- tri de la liste des étudiants par nom en ordre alphabétique.

Objectif pédagogique : Ce TP permet d'acquérir les bases du développement d'une API REST avec Spring Boot en mettant l'accent sur l'architecture en couches (Controller, Service, Repository), la persistance avec JPA, la gestion manuelle des erreurs HTTP et la documentation des services REST. Les annotations de validation (@Valid, @NotBlank, @Email, etc.) seront introduites dans un TP ultérieur.